```
# Import necessary libraries
import cv2
import matplotlib.pyplot as plt
from google.colab import files

# Upload the image to Colab
uploaded = files.upload()

# Load the Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# Read the uploaded image (you can change the filename if needed)
image = cv2.imread("z.jpg")

# Convert the image to grayscale (necessary for face detection)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces in the grayscale image
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Convert the BGR image (OpenCV format) to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display the image with the detected faces
plt.figure(figsize=(10, 10))
plt.imshow(image_rgb)
plt.axis('off')  # Hide the axis
plt.show()

# Print the number of faces detected
print(f"Number of faces detected: {len(faces)}")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving z.jpg to z (1).jpg
```

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files

# Upload the image to Colab
uploaded = files.upload()
```

```
# Load a different Haar Cascade (try other cascade models)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_alt2.xml')

# Read the uploaded image
image = cv2.imread("a.jpg")

# Convert the image to grayscale (necessary for face detection)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces in the grayscale image with tweaked parameters
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.2, minNeighbors=4, minSize=(30, 30))

# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Convert the BGR image (OpenCV format) to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display the image with the detected faces
plt.figure(figsize=(10, 10))
plt.imshow(image_rgb)
plt.axis('off')  # Hide the axis
plt.show()

# Print the number of faces detected
print(f"Number of faces detected: {len(faces)}")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving a.jpg to a.jpg
```

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
uploaded = files.upload()

# Load cascades for face and eyes
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

# Get uploaded filename
filename = next(iter(uploaded))

# Read image
```

```
image = cv2.imread(filename)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Detect faces normally
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

# Draw normal face detection results (in red)
image_faces = image.copy()
for (x, y, w, h) in faces:
    cv2.rectangle(image_faces, (x, y), (x + w, y + h), (0, 0, 255), 2)

# Face detection with eye verification:
faces_with_eyes = []
for (x, y, w, h) in faces:
    roi_gray = gray[y:y+h, x:x+w]
    detected_eyes = eye_cascade.detectMultiScale(roi_gray)
    if len(detected_eyes) >= 2:  # Face must have at least two eyes detected
        faces_with_eyes.append((x, y, w, h))

# Draw face detections verified by eyes (in green)
image_verified = image.copy()
for (x, y, w, h) in faces_with_eyes:
    cv2.rectangle(image_verified, (x, y), (x + w, y + h), (0, 255, 0), 3)

# Show images for comparison
fig, axs = plt.subplots(1, 2, figsize=(15, 10))

axs[0].imshow(cv2.cvtColor(image_faces, cv2.COLOR_BGR2RGB))
axs[0].set_title('Face Detection Only (Red boxes)')
axs[0].axis('off')

axs[1].imshow(cv2.cvtColor(image_verified, cv2.COLOR_BGR2RGB))
axs[1].set_title('Face Detection with Eye Verification (Green boxes)')
axs[1].axis('off')

plt.show()

print(f"Faces detected without eye check: {len(faces)}")
print(f"Faces detected with eye verification: {len(faces_with_eyes)}")
```

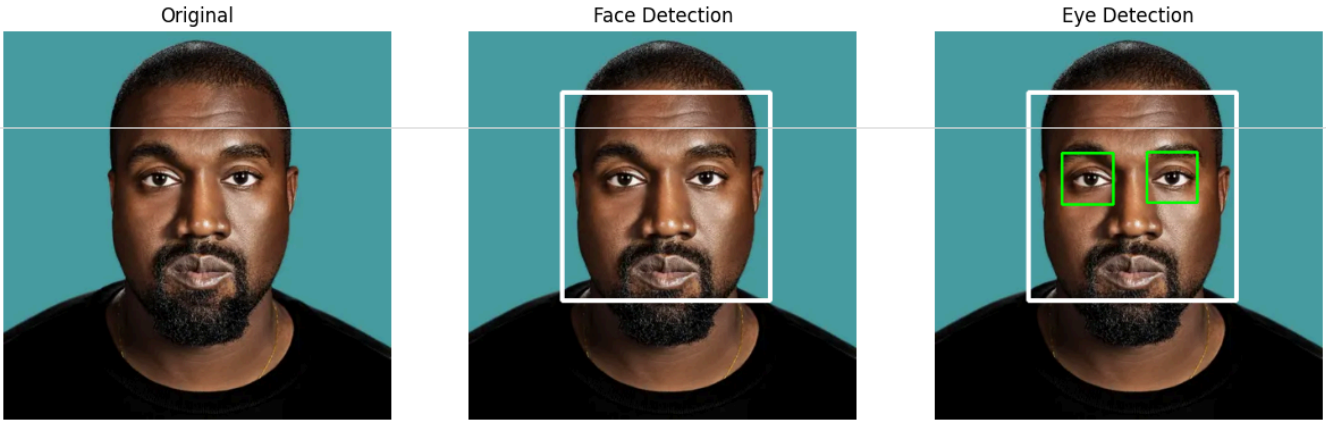
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving a.jpg to a (1).jpg
```

```
Faces detected without eye check: 5
Eyes detected: 0
Faces detected with eye verification: 0
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image from your local machine
uploaded = files.upload()

# Load cascades from OpenCV's default location
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
```

```
def adjusted_detect_face(img):
    face_img = img.copy()
    face_rect = face_cascade.detectMultiScale(face_img, scaleFactor=1.2, minNeighbors=5)
    for (x, y, w, h) in face_rect:
        cv2.rectangle(face_img, (x, y), (x + w, y + h), (255, 255, 255), 3)
    return face_img

def detect_face_and_eyes(img):
    img_copy = img.copy()
    gray = cv2.cvtColor(img_copy, cv2.COLOR_BGR2GRAY)

    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.2, minNeighbors=5)

    for (x, y, w, h) in faces:
        cv2.rectangle(img_copy, (x, y), (x + w, y + h), (255, 255, 255), 3)

        # Focus only on the face region
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = img_copy[y:y+h, x:x+w]

        eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1, minNeighbors=10, minSize=(30,30))
        for (ex, ey, ew, eh) in eyes:
            cv2.rectangle(roi_color, (ex, ey), (ex + ew, ey + eh), (0, 255, 0), 2)

    return img_copy

# Get the uploaded filename (assuming only one file uploaded)
filename = next(iter(uploaded))

# Read the uploaded image
img = cv2.imread(filename)

if img is None:
    print("Error: Image not loaded. Check the file path and format.")
else:
    face = adjusted_detect_face(img)
    eyes = detect_face_and_eyes(img)

    # Show all three in one figure
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title("Original")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.imshow(cv2.cvtColor(face, cv2.COLOR_BGR2RGB))
    plt.title("Face Detection")
    plt.axis("off")

    plt.subplot(1, 3, 3)
    plt.imshow(cv2.cvtColor(eyes, cv2.COLOR_BGR2RGB))
    plt.title("Eye Detection")
    plt.axis("off")

    plt.show()

    # Save results if needed
    cv2.imwrite("face.jpg", face)
    cv2.imwrite("eyes.jpg", eyes)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving KanyeWest.jpg to KanyeWest.jpg
```